ALOBALL

PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

Master Blueprint – Nền tảng quản lý và kết nối bóng đá phủi

Tài liệu định hướng triển khai từ ý tưởng đến MVP, dành cho phát triển bằng ASP.NET Core, React và Antigravity.

Phiên bản 1.0 | Ngày 02/11/2026 | Trạng thái: Draft để xác nhận trước khi code

Thông tin tài liệu

Thuộc tính	Nội dung
Tên sản phẩm	Aloball
Mục tiêu	Quản lý CLB bóng đá phủi, tổ chức trận đấu, tìm đối giao hữu và tạo động lực hoạt động.
Đối tượng	Lãnh đội, cầu thủ, đội bóng nghiệp dư, người xem công khai; quản trị viên nền tảng.
Phạm vi tài liệu	Vision, yêu cầu chức năng, quy trình nghiệp vụ, kiến trúc, dữ liệu, API, bảo mật, kiểm thử, triển khai và roadmap.
Định hướng kỹ thuật	ASP.NET Core Web API + EF Core + SQL Server; React + TypeScript; triển khai từng module.
Nguyên tắc	MVP trước; phân quyền ở Backend; dữ liệu trận đấu là nguồn gốc thống kê; coin có sổ giao dịch; AI không tự ý thay đổi kiến trúc.

Cách sử dụng tài liệu

- Đọc toàn bộ tài liệu trước khi yêu cầu AI sinh code.
- Xác nhận các quyết định đang đánh dấu TBD (cần quyết định).
- Dùng tài liệu làm nguồn chuẩn trong thư mục docs/ của repository.
- Mỗi lần triển khai chỉ giao AI một task nhỏ; cập nhật tài liệu khi quyết định thay đổi.

Mục lục

- 1. Tổng quan và mục tiêu sản phẩm
- 2. Phạm vi MVP và roadmap
- 3. Vai trò và phân quyền
- 4. Yêu cầu chức năng
- 5. Quy trình nghiệp vụ
- 6. Kiến trúc hệ thống
- 7. Thiết kế dữ liệu
- 8. API đề xuất
- 9. Quy tắc nghiệp vụ quan trọng
- 10. Bảo mật, riêng tư và kiểm soát rủi ro
- 11. UX, sitemap và trạng thái giao diện
- 12. Chiến lược kiểm thử
- 13. CI/CD và triển khai
- 14. Kế hoạch vibe coding với Antigravity
- 15. Definition of Done và checklist
- 16. Các quyết định cần xác nhận

1. Tổng quan và mục tiêu sản phẩm

Aloball là nền tảng web giúp các đội bóng đá phủi quản lý thành viên, lịch thi đấu, kết quả, thống kê và kết nối với đội bóng khác. Sản phẩm hướng tới trải nghiệm chuyên nghiệp nhưng đủ đơn giản để lãnh đội và cầu thủ sử dụng thường xuyên.

1.1. Tầm nhìn

Trở thành nền tảng số hỗ trợ hoạt động thường ngày của cộng đồng bóng đá nghiệp dư: từ tìm đội, quản lý đội, lên lịch đá, ghi nhận trận đấu đến tìm đối giao hữu.

1.2. Giá trị cốt lõi

- Quản lý đội bóng tập trung, giảm việc ghi chép thủ công qua chat và bảng tính.
- Thông tin đội/cầu thủ có trang public được kiểm soát bởi quyền riêng tư.
- Tìm đối giao hữu dựa trên khu vực, lịch, sân và trình độ phù hợp.
- Ghi nhận lịch sử trận, bàn thắng, kiến tạo, chuyên cần và thành tích.
- Tạo động lực tham gia bằng AB Coin, danh hiệu và bảng xếp hạng.

1.3. Đối tượng người dùng

Persona	Nhu cầu chính	Thành công khi
Khán giả / Guest	Xem đội bóng, lịch, thành tích và hồ sơ public.	Tìm được thông tin đáng tin mà không cần đăng nhập.
Cầu thủ / Player	Xem lịch, xác nhận tham gia, theo dõi thống kê và coin.	Biết lịch đội, lịch sử và thành tích cá nhân.
Lãnh đội / Captain	Quản lý CLB, thành viên, trận, điểm danh, tìm đối.	Tạo và quản lý trận nhanh, dữ liệu rõ ràng.
Admin	Kiểm duyệt, xử lý báo cáo, quản lý tài khoản và đội.	Giữ nền tảng an toàn, công bằng và đáng tin.

1.4. Nguyên tắc sản phẩm

- Public-first nhưng privacy-aware: thông tin công khai phải có cấu hình/đồng ý phù hợp.
- Captain là quyền theo từng đội, không phải quyền toàn hệ thống.
- Không tin dữ liệu do frontend gửi; Backend xác thực và kiểm tra quyền.
- Kết quả trận đấu là dữ liệu gốc cho thống kê và AB Coin.
- Các thao tác nhạy cảm cần audit log và có thể sửa sai có kiểm soát.

2. Phạm vi MVP và roadmap

2.1. MVP – Aloball 1.0

Ưu tiên	Module	Phạm vi MVP
P0	Authentication & Profile	Đăng ký/đăng nhập, hồ sơ cơ bản, role Player/Admin, OAuth sau khi email auth ổn định.
P0	Teams	Tạo đội, hồ sơ đội, thành viên, Captain/Co-captain, trang public.
P0	Matches	Tạo trận giao hữu, lịch, địa điểm, đội hình và trạng thái trận.
P0	Attendance	Cầu thủ xác nhận tham gia; Captain chốt điểm danh.
P0	Match Results	Tỉ số, goal, assist, xác nhận kết quả và lịch sử.

Ưu tiên	Module	Phạm vi MVP
P1	AB Coin	Coin attendance/goal/assist/win; transaction ledger và chống cộng trùng.
P1	Opponent matching	Đăng tin tìm đối, gửi/nhận lời mời, chốt trận.
P1	Public pages	Trang đội, cầu thủ, lịch và thành tích public với trường dữ liệu giới hạn.

2.2. Version 2

- Rating hai đội sau trận; tiêu chí fair-play, đúng giờ, giao tiếp và tổ chức.
- Cơ chế báo cáo, khiếu nại, cảnh báo uy tín và blacklist do admin xét duyệt.
- Keep the Fire: chuỗi chuyên cần, danh hiệu tháng, MVP, vua phá lưới/kiến tạo.
- Tìm đội và tuyển cầu thủ; profile matching theo khu vực, vị trí và tần suất.
- Mini game penalty và đố vui; không tạo coin farming hoặc ảnh hưởng dữ liệu trận thật.

2.3. Version 3

- Tạo giải đấu, đăng ký đội, bảng đấu, lịch và standings.
- Lịch sử thành tích đội qua mùa giải.
- Tích hợp đặt sân, thông báo realtime, push notification.
- Thống kê nâng cao, API tích hợp và các tính năng cộng đồng.

2.4. Ngoài phạm vi MVP

- Coin chuyển nhượng giữa người dùng hoặc quy đổi thành tiền.
- Tự động blacklist chỉ từ một rating 1 sao.
- Tổ chức giải đấu đầy đủ, bracket phức tạp.
- Mini game có thưởng coin lớn.
- Ứng dụng mobile native riêng.

3. Vai trò và phân quyền

3.1. Mô hình quyền

Dùng ASP.NET Core Identity cho tài khoản và role nền tảng (Admin). Quyền Captain/Player/Co-captain được quản lý theo quan hệ TeamMember trong từng đội. Một người có thể là Captain đội A và Player đội B.

Vai trò	Phạm vi	Quyền chính
Guest	Toàn hệ thống public	Xem trang public; không sửa dữ liệu.
Player	Tài khoản cá nhân	Sửa hồ sơ của mình; xem lịch đội; xác nhận tham gia; gửi lời mời/đăng ký theo quy tắc.
Captain	Một Team cụ thể	Quản lý thông tin đội, thành viên, trận, đội hình, điểm danh và kết quả trong phạm vi đội.
Co-captain (V2)	Một Team cụ thể	Quyền được Captain/Admin cấp; có thể giới hạn theo tác vụ.
Admin	Toàn hệ thống	Quản lý người dùng, xử lý báo cáo, kiểm duyệt, khóa tài khoản/đội, audit.

3.2. Quy tắc phân quyền

- Không dùng role Captain toàn cục để cấp quyền mọi đội.
- Mỗi endpoint thay đổi dữ liệu phải kiểm tra membership và role của TeamId cụ thể.

- Frontend chỉ ẩn/hiện nút; Backend mới là nơi quyết định quyền.
- Mỗi thay đổi kết quả trận hoặc coin cần ghi actor, thời gian và lý do nếu sửa.
- Captain không được tự sửa dữ liệu đội khác, kể cả khi biết ID.

3.3. Ma trận quyền MVP

Tác vụ	Guest	Player	Captain của đội	Admin
Xem đội public	Có	Có	Có	Có
Sửa hồ sơ cá nhân	Không	Chỉ mình	Chỉ mình	Có/điều chỉnh
Tạo đội	Không	Có	Có	Có
Sửa đội	Không	Không	Đội mình	Có
Quản lý thành viên	Không	Không	Đội mình	Có
Tạo/sửa trận	Không	Theo quyền tham gia	Đội mình	Có
Chốt kết quả	Không	Không	Đội mình, có kiểm soát	Có
Xử lý báo cáo/blacklist	Không	Không	Không	Có

4. Yêu cầu chức năng

4.1. Tài khoản và hồ sơ

- Đăng ký email/password, xác thực email, đăng nhập, đăng xuất và quên mật khẩu.
- OAuth Google/Facebook/Apple là tính năng bổ sung; cần kiểm tra provider, callback, chính sách và secrets.
- Profile: display name, avatar, vị trí chính/phụ, chân thuận, kinh nghiệm, khu vực, đội hiện tại.
- Ngày sinh, chiều cao, cân nặng và số điện thoại không public mặc định; người dùng kiểm soát visibility.
- Cho phép cập nhật profile và kiểm tra dữ liệu hợp lệ.

4.2. Đội bóng

- Tạo đội: tên, logo, ngày thành lập, mô tả, khu vực, sân nhà, phạm vi hoạt động.
- Danh sách thành viên: tên, avatar, số áo, vị trí, trạng thái thành viên.
- Captain quản lý lời mời, thêm/xóa thành viên và số áo.
- Public team page hiển thị trường được phép công khai.
- Xuất danh sách đặt áo CSV/Excel/PDF là enhancement sau MVP.

4.3. Trận đấu

- Tạo trận nội bộ/giao hữu: thời gian, sân, địa chỉ, loại sân, đội tham gia, số người và ghi chú.
- Trạng thái: Draft → Scheduled → Confirmed → InProgress → Completed; Cancelled là trạng thái kết thúc riêng.
- Cầu thủ RSVP: Going / Maybe / NotGoing; Captain chốt attendance.
- Đội hình và danh sách tham gia được lưu theo từng trận, không chỉ tham chiếu roster hiện tại.
- Kết quả gồm score, goal, assist, phút ghi bàn (optional), người ghi nhận.
- Cho phép chỉnh kết quả có audit log và kiểm tra tính hợp lệ.

4.4. Tìm đối

- Captain đăng tin: ngày, khung giờ, khu vực, sân nhà/đi khách, trình độ, số người, chi phí/điều kiện.
- Đội khác tìm kiếm/lọc theo khu vực, thời gian, trình độ và loại sân.
- Gửi lời mời; đội chủ tin accept/reject; khi accept tạo/hoặc liên kết Match.
- Chặn lời mời trùng lịch nếu dữ liệu lịch đã xác nhận.
- Tin tìm đối có trạng thái Open, Pending, Accepted, Closed, Expired, Cancelled.

4.5. Kết quả và thống kê

- Goal/Assist được ghi theo MatchParticipant/Player và MatchId.
- Thống kê trận, tổng bàn, kiến tạo, số trận, win/draw/loss, attendance.
- Tính từ dữ liệu trận đã Completed và đã xác nhận; không tính trận bị hủy.
- Lịch sử trận hiển thị đội hình và kết quả theo quyền truy cập.

4.6. AB Coin & Keep the Fire

Sự kiện	Coin gợi ý	Điều kiện
Attendance	+2	Chỉ khi Captain chốt có mặt; mỗi cầu thủ/mỗi trận tối đa một lần.
Goal	+3	Goal event hợp lệ; mỗi event chỉ thưởng một lần.
Assist	+1	Assist event hợp lệ; chống ghi trùng.
Team win	+1	Mỗi cầu thủ đủ điều kiện tham gia trận; chỉ sau khi kết quả xác nhận.

Coin là điểm ảo nội bộ, không phải tiền và không quy đổi tiền thật trong MVP. Tất cả thay đổi coin phải ghi vào CoinTransactions. Sử dụng idempotency/unique constraints để tránh cộng thưởng lặp khi retry.

4.7. Rating, report và blacklist (V2)

- Rating chỉ được tạo bởi đội đã có trận hợp lệ với nhau.
- Mỗi cặp đội/trận chỉ có rating theo quy định; rating có thể gồm điểm sao và comment có kiểm duyệt.
- Một rating thấp không tự động blacklist.
- Admin xem xét báo cáo, lịch sử, bằng chứng và quyết định cảnh báo/tạm khóa.
- Blacklist chỉ ảnh hưởng tính năng tìm đối/gửi lời mời; không khóa quản lý nội bộ đội.

5. Quy trình nghiệp vụ

5.1. Onboarding

1. Người dùng đăng ký/đăng nhập.
2. Chọn mục đích: khán giả, cầu thủ, lãnh đội (có thể chọn nhiều theo hành động, không khóa cứng profile).
3. Cầu thủ nhập vị trí, khu vực, kinh nghiệm, tần suất thi đấu và trạng thái có đội/chưa có đội.
4. Nếu muốn làm Captain, người dùng tạo đội hoặc nhận quyền Captain từ đội hiện có.
5. Hệ thống đưa người dùng tới dashboard phù hợp.

5.2. Tạo đội và quản lý roster

1. Player tạo Team.

2. Hệ thống tạo TeamMember record cho người tạo với role Captain.
3. Captain cập nhật logo, thông tin và visibility.
4. Captain mời thành viên bằng link/email hoặc tìm tài khoản.
5. Thành viên chấp nhận lời mời; TeamMember chuyển Active.

5.3. Tổ chức trận giao hữu nội bộ

1. Captain tạo Match và lịch.
2. Hệ thống gửi thông báo hoặc hiển thị trên calendar.
3. Cầu thủ RSVP.
4. Captain chốt roster/attendance trước hoặc sau trận.
5. Trận diễn ra; Captain ghi tỉ số và events.
6. Kết quả được xác nhận; Match chuyển Completed.
7. Backend xử lý coin và cập nhật thống kê trong transaction/idempotent workflow.

5.4. Bắt đối

1. Team A đăng OpponentRequest.
2. Team B tìm kiếm và gửi MatchInvitation.
3. Team A accept; hệ thống kiểm tra trạng thái, lịch và quyền.
4. Tạo Match liên kết request/invitation; hai đội xác nhận.
5. Sau trận, hai bên hoàn tất kết quả và (V2) rating.

5.5. Chốt kết quả và sửa sai

- MVP: Captain của đội chủ trận nhập kết quả; Captain đội còn lại xác nhận hoặc Admin can thiệp.
- Nếu hai bên không đồng thuận, Match có trạng thái ResultDisputed (có thể V2 nếu MVP cần đơn giản).
- Mọi sửa score/events sau khi coin đã phát phải tạo adjustment ledger; không xóa giao dịch lịch sử.
- Không để client tự gửi tổng coin hoặc tự quyết định đội thắng.

6. Kiến trúc hệ thống

6.1. Kiến trúc đề xuất

MVP dùng Modular Monolith: một Backend deployable, chia module theo nghiệp vụ. Không cần microservices ở giai đoạn đầu.

Layer/Project	Trách nhiệm
Aloball.API	HTTP endpoints, authentication middleware, request/response, Swagger, DI, configuration.
Aloball.Application	Use cases, DTOs, validation, interfaces, business orchestration.
Aloball.Domain	Entities, enums, domain rules và các quy tắc độc lập framework.
Aloball.Infrastructure	EF Core DbContext, migrations, repositories nếu cần, Identity, email/storage provider.
Aloball.Tests	Unit tests và integration tests.

6.2. Công nghệ

Thành phần	Khuyến nghị	Ghi chú
Backend	ASP.NET Core Web API (.NET 10 LTS)	Kiểm tra SDK/runtime và hosting hỗ trợ trước khi chọn.
ORM	EF Core phiên bản tương thích .NET	Migration, LINQ, tracking và transaction.
Database	SQL Server	Local dev có thể dùng SQL Server Developer/Express hoặc container phù hợp.
Frontend	React + TypeScript + Vite	SPA; gọi API qua typed service layer.
UI	Tailwind CSS + component library đã chọn	Chọn một hệ thống UI nhất quán, tránh mix nhiều bộ.
API docs	Swagger/OpenAPI	Dùng để kiểm tra API và hỗ trợ frontend.
Tests	xUnit + integration test	Test nghiệp vụ và endpoint/authorization.
Version control	Git + GitHub	Branch, commit nhỏ, PR hoặc review.

6.3. Cấu trúc repository

```
Aloball/
├── Aloball.sln
├── docs/
│   ├── PRODUCT_REQUIREMENTS.md
│   ├── SYSTEM_ARCHITECTURE.md
│   ├── DATABASE_DESIGN.md
│   ├── API_SPECIFICATION.md
│   ├── AUTHORIZATION_RULES.md
│   ├── ADR/
│   └── prompts/
├── src/
│   ├── Aloball.API/
│   ├── Aloball.Application/
│   ├── Aloball.Domain/
│   └── Aloball.Infrastructure/
├── tests/
│   ├── Aloball.UnitTests/
│   └── Aloball.IntegrationTests/
├── frontend/
│   └── aloball-web/
├── .gitignore
├── README.md
└── docker-compose.yml (optional for local dev)
```

6.4. Quy tắc code

- Controller mỏng; business logic nằm trong Application/domain service.
- Request/response dùng DTO; không serialize EF entity trực tiếp.
- Async/await cho I/O; cancellation token ở endpoint/service phù hợp.
- Validation server-side; frontend validation chỉ hỗ trợ UX.
- Không lạm dụng repository pattern nếu DbContext đã đủ; quyết định nhất quán.
- Không triển khai CQRS/MediatR chỉ vì AI đề xuất; chỉ thêm khi có lý do.

7. Thiết kế dữ liệu

Thiết kế dưới đây là logical schema khởi đầu. Cần review trước migration đầu tiên. Dùng khóa chính dạng GUID hoặc bigint nhất quán; đề xuất GUID cho các ID public-facing và lưu CreatedAt/UpdatedAt UTC.

7.1. Entity chính

Entity	Các trường quan trọng	Ghi chú
ApplicationUser	Id, UserName, Email, DisplayName, Status	ASP.NET Core Identity; không duplicate password.
PlayerProfile	UserId, AvatarUrl, PrimaryPosition, SecondaryPosition, PreferredFoot, Experience, RegionId, Visibility	1-1 với User; privacy settings.
Team	Id, Name, Slug, LogoUrl, FoundedAt, Description, RegionId, HomeVenueId, ActivityScope, Status, CreatedByUserId	Public team profile.
TeamMember	Id, TeamId, UserId, TeamRole, JerseyNumber, Position, JoinedAt, LeftAt, Status	Unique active membership theo quy tắc; role scoped to team.
Venue	Id, Name, Address, RegionId, SurfaceType, IsPublic	Địa điểm/sân; không nhất thiết MVP cần quản lý đầy đủ.
Match	Id, MatchType, Status, StartAtUtc, EndAtUtc, VenueId, HomeTeamId, AwayTeamId, CreatedByUserId, ResultStatus	MatchType: Internal/Friendly; nullable AwayTeam trong trận nội bộ nếu cần.
MatchParticipant	Id, MatchId, TeamId, UserId, JerseyNumberSnapshot, PositionSnapshot, RSVPStatus, AttendanceStatus	Snapshot để roster thay đổi không làm sai lịch sử trận.
MatchEvent	Id, MatchId, TeamId, PlayerUserId, EventType, Minute, AssistUserId, CreatedByUserId, IsVoided	Goal/Assist; validate participant belongs to match.
OpponentRequest	Id, TeamId, DateFromUtc, DateToUtc, RegionId, VenuePreference, SkillLevel, PlayerCount, Notes, Status, ExpiresAtUtc	Tin tìm đối.
MatchInvitation	Id, OpponentRequestId, InvitingTeamId, ReceivingTeamId, Status, ProposedMatchAtUtc	Chống duplicate invitation.
CoinTransaction	Id, UserId, MatchId, Amount, Type, IdempotencyKey, CreatedByUserId, CreatedAtUtc, ReversalOfTransactionId	Ledger append-only; không xóa lịch sử.
TeamRating (V2)	Id, MatchId, RaterTeamId, RatedTeamId, Stars, CriteriaJson/columns, Comment, Status	Chỉ sau trận hợp lệ; moderation.
AuditLog	Id, ActorUserId, EntityType, EntityId, Action, BeforeJson, AfterJson, CreatedAtUtc	Lưu thao tác nhạy cảm, hạn chế dữ liệu nhạy cảm trong JSON.

7.2. Quan hệ chính

- ApplicationUser 1–1 PlayerProfile.
- ApplicationUser 1–N TeamMember; Team 1–N TeamMember.
- Team 1–N Match với vai trò HomeTeam/AwayTeam.
- Match 1–N MatchParticipant; MatchParticipant liên kết User và Team.
- Match 1–N MatchEvent; mỗi event tham chiếu cầu thủ trong participant.
- User 1–N CoinTransaction; Match có thể là nguồn phát sinh coin.
- OpponentRequest thuộc Team; MatchInvitation nối request và hai đội.

7.3. Constraint và index quan trọng

- Unique TeamMember active theo (TeamId, UserId) hoặc quy tắc membership được xác định rõ.
- Unique CoinTransaction.IdempotencyKey; unique theo source event/reward type để chống cộng trùng.
- Index Match theo HomeTeamId/AwayTeamId/StartAtUtc/Status.
- Index OpponentRequest theo RegionId/DateFromUtc/Status/ExpiresAtUtc.
- Foreign key và delete behavior rõ ràng; tránh cascade delete các trận/lịch sử.
- JerseyNumber unique trong cùng đội nếu đội quy định; nullable nếu chưa cấp.

- Lưu thời gian UTC; chuyển timezone tại frontend theo người dùng/sân.

7.4. Lưu ý thống kê

Không lưu mọi tổng số bàn/thắng/coin như dữ liệu nguồn độc lập. MatchEvent và CoinTransaction là lịch sử gốc; các bảng tổng hợp/cached stats chỉ thêm khi có nhu cầu hiệu năng và phải có cơ chế rebuild.

8. API đề xuất

API dưới đây là bản nháp REST naming. Chốt request/response, pagination, error format và versioning trước khi code frontend.

Nhóm	Endpoint ví dụ	Quyền
Auth	POST /api/auth/register; POST /api/auth/login; POST /api/auth/logout; POST /api/auth/forgot-password	Public / current user
Profile	GET /api/me; PUT /api/me/profile; GET /api/players/{id}	Owner sửa; public fields theo visibility
Teams	GET /api/teams; POST /api/teams; GET /api/teams/{id}; PUT /api/teams/{id}; DELETE/archive	Public read; Captain team sửa
Team members	GET /api/teams/{id}/members; POST /api/teams/{id}/invitations; PATCH /api/teams/{id}/members/{memberId}	Captain / member
Matches	GET /api/matches; POST /api/matches; GET /api/matches/{id}; PATCH /api/matches/{id}; POST /api/matches/{id}/cancel	Participant/Captain
Attendance	PUT /api/matches/{id}/attendance/me; POST /api/matches/{id}/attendance/close	Player RSVP; Captain chốt
Events	POST /api/matches/{id}/events; PATCH /api/matches/{id}/events/{eventId}; DELETE/void event	Captain hoặc scorer được cấp quyền
Results	POST /api/matches/{id}/result/submit; POST /api/matches/{id}/result/confirm; POST /api/matches/{id}/result/dispute	Captain hai đội / Admin
Opponent	GET /api/opponent-requests; POST /api/opponent-requests; POST /api/opponent-requests/{id}/invitations; POST /api/invitations/{id}/accept	Captain
Coin	GET /api/me/coins; GET /api/me/coin-transactions	Read own; coin changes server-only
Admin	GET /api/admin/reports; POST /api/admin/teams/{id}/suspend; POST /api/admin/users/{id}/suspend	Admin

8.1. Quy chuẩn API

- Dùng DTO riêng cho request/response.
- Phân trang danh sách: page/pageSize hoặc cursor; giới hạn pageSize.
- Validation lỗi trả 400; chưa đăng nhập 401; không đủ quyền 403; không tìm thấy 404; conflict nghiệp vụ 409.
- Không trả stack trace hoặc thông tin nhạy cảm cho client.
- Swagger có mô tả, schema, response codes và auth scheme.
- Không dùng GET để thay đổi dữ liệu.

9. Quy tắc nghiệp vụ quan trọng

9.1. Trận đấu

- Match Completed phải có kết quả hợp lệ; trận Cancelled không phát coin.
- Không cho sửa roster snapshot của trận sau khi trận hoàn tất, trừ workflow sửa có audit.
- Một cầu thủ chỉ có một attendance record cho mỗi MatchParticipant.
- Goal/Assist phải gắn với cầu thủ thuộc trận; assist không được tự động là goal scorer.
- Trận giữa hai đội phải có hai team khác nhau; đội hình không vượt số lượng cho phép nếu có cấu hình.

9.2. AB Coin

- Coin được cộng ở Backend sau khi điều kiện nghiệp vụ được xác nhận.
- Mỗi loại thưởng phải có idempotency key duy nhất.
- Không cho client truyền Amount cuối cùng; client chỉ gửi action/event hợp lệ.
- Coin correction dùng reversal/adjustment transaction, không xóa ledger.
- MVP coin chỉ là điểm ảo, không mua bán, không chuyển giữa người dùng.

9.3. Rating và blacklist

- Rating phải dựa trên Match đã Completed và đội có tham gia.
- Rating có thể bị ẩn trong thời gian review nếu có báo cáo.
- Blacklist do Admin quyết định dựa trên quy trình và lý do.
- Blacklist không xóa dữ liệu đội hoặc khóa lịch nội bộ.
- Có thời hạn, trạng thái và quy trình kháng nghị nếu triển khai.

9.4. Quyền sở hữu dữ liệu

- Captain chỉ sửa dữ liệu Team mà họ đang giữ quyền Captain.
- Người dùng chỉ sửa profile của chính mình, ngoại trừ Admin theo chính sách.
- Khi Captain rời đội, phải có quy trình chuyển quyền để tránh đội không còn người quản lý.
- Khi user bị xóa/đóng tài khoản, lịch sử trận và coin phải xử lý theo retention policy; không cascade xóa dữ liệu lịch sử tùy tiện.

10. Bảo mật, riêng tư và kiểm soát rủi ro

10.1. Bảo mật cơ bản

- Dùng ASP.NET Core Identity hoặc provider đã được kiểm chứng; không tự lưu password.
- HTTPS ở môi trường production; secrets không commit vào Git.
- Authorization server-side cho mọi resource theo TeamId/ownership.
- Rate limit login, register, password reset và endpoint public dễ bị abuse.
- Validate file upload: loại file, kích thước, tên file, storage; không tin extension.
- CORS chỉ cho phép frontend domain cần thiết.
- Không ghi password, token, email nhạy cảm hoặc secret vào logs.
- Dùng anti-forgery/cookie security nếu chọn cookie auth; nếu bearer token, quản lý token storage và refresh theo threat model.

10.2. Privacy

- Email, số điện thoại, ngày sinh đầy đủ, chiều cao/cân nặng private mặc định.
- Public profile chỉ trả những trường được phép hiển thị.
- Avatar/logo cần giới hạn file và kiểm soát nội dung.
- Cho phép người dùng ẩn thông tin cá nhân; cần có chính sách xử lý dữ liệu và xóa tài khoản.
- Không public thông tin vị trí chính xác của người dùng; chỉ khu vực hoạt động.

10.3. Abuse prevention

- Chống tạo đội/trận/tin tìm đối spam bằng rate limit và trạng thái tài khoản.
- Chống coin farming bằng validation server-side, idempotency và audit.
- Rating chỉ sau trận hợp lệ; report/comment có moderation.
- Admin action cần audit trail.
- Có quy trình xử lý tranh chấp kết quả và khiếu nại.

11. UX, sitemap và trạng thái giao diện

11.1. Sitemap

```
Public:
- /
- /teams
- /teams/{slug}
- /players/{id}
- /matches
- /opponents
- /tournaments (V3)

Authenticated:
- /app/dashboard
- /app/profile
- /app/calendar
- /app/coins
- /app/teams/{teamId}
- /app/teams/{teamId}/members
- /app/teams/{teamId}/matches
- /app/teams/{teamId}/settings

Captain:
- /app/captain/teams/{teamId}/overview
- /app/captain/teams/{teamId}/match/new
- /app/captain/teams/{teamId}/attendance
- /app/captain/opponent-requests

Admin:
- /admin/users
- /admin/teams
- /admin/reports
- /admin/audit
```

11.2. UI states bắt buộc

- Loading/skeleton.
- Empty state có hướng dẫn hành động.
- Error state và retry.
- Success feedback.
- Permission denied.
- Form validation và dirty state.

- Responsive mobile/tablet/desktop.
- Confirm dialog cho xóa, hủy trận, sửa kết quả.

11.3. Design direction

Phong cách thể thao hiện đại, rõ thông tin: nền trung tính, màu accent xanh lá hoặc xanh dương, typography dễ đọc, số liệu và trạng thái nổi bật. Tránh biến dashboard thành trang quá nhiều card/gradient. Chọn một design system và dùng nhất quán.

12. Chiến lược kiểm thử

12.1. Unit tests

- AB Coin calculation: attendance, goal, assist, win và điều kiện không thưởng.
- Match state transitions.
- Authorization policy/team role.
- Rating eligibility.
- Opponent request state transitions.

12.2. Integration tests

- API tạo đội và tự gán Captain.
- Player không thể sửa Team của người khác.
- Captain đội A không thể sửa Team B.
- Tạo match và participant snapshot.
- Hoàn tất trận phát coin đúng một lần.
- Retry request không tạo coin duplicate.
- Public endpoint không lộ email/private fields.
- Migration chạy trên database test.

12.3. Manual acceptance scenarios

1. Người mới đăng ký và tạo profile.
2. Người dùng tạo đội và trở thành Captain.
3. Captain mời thành viên; thành viên chấp nhận.
4. Captain tạo trận; cầu thủ RSVP; Captain chốt attendance.
5. Captain ghi bàn/kiến tạo và kết quả.
6. Backend phát coin đúng quy tắc.
7. Đội khác đăng tin tìm đối và gửi lời mời.
8. Guest xem public team nhưng không thấy thông tin private.
9. Người không có quyền thử gọi API sửa dữ liệu và nhận 403.

13. CI/CD và triển khai

13.1. Môi trường

Môi trường	Mục đích	Cấu hình
Local	Code/test	Local SQL Server; secrets qua user-secrets/.env; frontend dev server.
Development/Staging	Test tích hợp và demo	Database riêng, HTTPS, test account, không dùng dữ liệu cá nhân thật.
Production	Người dùng thật	HTTPS, secret manager/environment variables, backup, logging, monitoring.

13.2. CI pipeline tối thiểu

1. Restore dependencies.
2. Build Backend và Frontend.
3. Chạy unit tests.
4. Chạy lint/typecheck frontend.
5. Chạy integration tests với database test nếu pipeline đã sẵn sàng.
6. Build artifact/container.
7. Deploy staging; migration có kiểm soát.
8. Smoke test trước khi promote production.

13.3. Production checklist

- HTTPS và domain.
- Database backup/restore test.
- Secrets không nằm trong repository.
- Logging/monitoring và error tracking.
- CORS, rate limits, file upload policy.
- Migration strategy và rollback plan.
- Privacy policy/terms/contact nếu mở public.
- Admin account bảo mật mạnh; không seed mật khẩu mặc định.

14. Kế hoạch vibe coding với Antigravity

14.1. Nguyên tắc làm việc

- Antigravity là coding agent, không phải người quyết định kiến trúc thay bạn.
- Mỗi prompt chỉ có một mục tiêu có thể kiểm tra.
- Trước khi sửa code: agent đọc docs và tóm tắt kế hoạch.
- Không cho agent tự ý refactor toàn repository.
- Sau mỗi task: build, test, inspect diff và commit.
- Không chấp nhận code bạn không thể giải thích ở mức cơ bản.

14.2. Phân chia công việc

Bạn tự làm / phải hiểu	AI có thể làm nhanh	AI tạo, bạn review
Entity design và quan hệ	UI layout, responsive, CSS	Identity/OAuth setup

Bạn tự làm / phải hiểu	AI có thể làm nhanh	AI tạo, bạn review
DTO/API contract	Form, cards, navigation	EF migrations
Business rules coin/match	Mock data và seed draft	Upload/storage
Authorization policy	README draft, Swagger descriptions	Docker/CI config
LINQ/EF queries quan trọng	UI states và accessibility	Tests draft, logging
Unit test logic cốt lõi	Refactor UI component	Dependency/package choices

14.3. Prompt chuẩn cho mỗi task

```
TASK: [Tên task cụ thể]

Context:
- Đọc docs/PRODUCT_REQUIREMENTS.md
- Đọc docs/SYSTEM_ARCHITECTURE.md
- Đọc docs/DATABASE_DESIGN.md
- Đọc docs/AUTHORIZATION_RULES.md

Goal:
[Chỉ một mục tiêu]

Constraints:
- Không thay đổi kiến trúc ngoài phạm vi.
- Không sửa module không liên quan.
- DTO riêng; business logic không nằm trong Controller.
- Authorization server-side.
- Không thêm package nếu chưa giải thích.

Before coding:
1. Tóm tắt hiểu biết.
2. Liệt kê file sẽ tạo/sửa.
3. Nêu edge cases và test plan.
4. Chờ xác nhận nếu có thay đổi schema/architecture.

After coding:
1. Build và chạy test.
2. Tóm tắt file đã thay đổi.
3. Nêu lỗi/giới hạn còn lại.
4. Giải thích 3–5 kiến thức .NET quan trọng.
5. Không tuyên bố hoàn thành nếu test chưa chạy.
```

14.4. Prompt khởi tạo tài liệu

```
Bạn là Software Architect cho Aloball. Hãy chỉ phân tích và tạo tài liệu trong thư mục docs:
- PRODUCT_REQUIREMENTS.md
- SYSTEM_ARCHITECTURE.md
- DATABASE_DESIGN.md
- API_SPECIFICATION.md
- AUTHORIZATION_RULES.md
- DEVELOPMENT_ROADMAP.md

Dùng tài liệu PDF Aloball System Analysis & Design Blueprint làm nguồn tham khảo.
Không viết code, không tạo migration, không cài package.
Liệt kê mọi giả định và câu hỏi chưa quyết định bằng TBD.
Kết thúc bằng checklist các quyết định cần tôi xác nhận.
```

14.5. Prompt tạo Backend foundation

```
TASK: Khởi tạo Aloball Backend Foundation

Đọc toàn bộ docs đã được xác nhận.
Tạo solution và các project API, Application, Domain, Infrastructure, UnitTests.
Thiết lập dependency direction đúng theo tài liệu.
Cấu hình EF Core/SQL Server, Identity theo phạm vi đã thống nhất, Swagger, logging và global error handling tối thiểu.
Không tạo toàn bộ entity/module nghiệp vụ.
Không triển khai OAuth providers ở task này.
Tạo README hướng dẫn chạy local.
```

```
Build solution và báo kết quả.
Giải thích project reference và dependency injection.
```

14.6. Prompt triển khai từng feature

```
TASK: Implement [Feature Name] – [một use case]

Đọc tài liệu feature và API spec.
Trước tiên hãy trình bày:
- Use case và quyền truy cập
- Request/response DTO
- Validation
- Business rules
- Database changes (nếu có)
- Unit/integration tests
- Các file dự kiến thay đổi

Sau khi tôi xác nhận, implement đúng phạm vi.
Không sửa feature khác.
Không dùng mock data trong Backend production code.
Không trả Entity trực tiếp.
Không tin dữ liệu role/team/coin từ client.
Chạy build và tests; báo chính xác test nào đã chạy.
```

14.7. Prompt review bảo mật

```
Review feature [name] như một Backend Security Reviewer.
Kiểm tra:
- Authentication/Authorization và IDOR theo TeamId
- Input validation
- Data exposure/private fields
- SQL/EF query risks
- Race conditions và duplicate requests
- Coin/event manipulation
- Logging/secrets
- Error handling
Không sửa code ngay. Trả về danh sách findings theo severity, file/line, tác động và phương án sửa. Chờ tôi xác nhận.
```

15. Definition of Done và checklist

Một feature chỉ được coi là hoàn thành khi:

- Yêu cầu và quyền truy cập được mô tả.
- DTO, validation và business rules rõ ràng.
- Backend authorization được kiểm tra.
- Database constraints/migration đã review.
- Unit test cho logic cốt lõi.
- Integration test cho API/quyền quan trọng.
- Frontend có loading/empty/error/success state.
- Build thành công; tests được chạy và ghi kết quả.
- Không có secret hoặc dữ liệu nhạy cảm trong Git/log.
- README/docs cập nhật nếu có thay đổi contract.
- Bạn có thể giải thích code và luồng dữ liệu.

15.1. Git workflow

1. Tạo branch feature/<name>.
2. Thực hiện task nhỏ.

3. Review diff và test.
4. Commit với message mô tả rõ.
5. Merge sau khi kiểm tra.
6. Không để AI thực hiện commit/push/merge hoặc deploy production mà chưa có bạn xác nhận.

16. Các quyết định cần xác nhận trước khi code

Các mục dưới đây ảnh hưởng trực tiếp đến thiết kế; hãy chốt trong docs/DECISIONS.md.

ID	Câu hỏi	Khuyến nghị ban đầu
D-01	Một cầu thủ có thể thuộc nhiều đội cùng lúc?	Có; membership theo Team.
D-02	Đội có thể có nhiều Captain?	Một Captain chính + Co-captain ở V2.
D-03	Ai chốt kết quả trận?	Captain đội chủ trận nhập; đội còn lại xác nhận; Admin xử lý tranh chấp.
D-04	AB Coin có chuyển nhượng/đổi tiền?	Không trong MVP; coin chỉ là điểm ảo.
D-05	Rating 1 sao có blacklist ngay?	Không; cần report/moderation và Admin review.
D-06	Public có hiển thị chiều cao/cân nặng?	Opt-in; private mặc định.
D-07	Frontend?	React + TypeScript + Vite; nếu muốn đơn giản hơn có thể Razor MVC.
D-08	Authentication?	Identity email/password trước; OAuth sau khi core ổn định.
D-09	Khu vực địa lý?	Dùng Region hierarchy có thể mở rộng; MVP bắt đầu tỉnh/thành + quận/huyện.
D-10	Sân bóng?	MVP lưu venue text/địa chỉ; entity Venue đầy đủ ở bước tiếp theo.

17. Bước bắt đầu ngay hôm nay

1. Tạo repository GitHub riêng: Aloball.
2. Tạo thư mục docs/ và lưu PDF này vào tài liệu dự án.
3. Mở Antigravity tại root repository.
4. Dùng prompt khởi tạo tài liệu ở mục 14.4; yêu cầu agent tạo docs, chưa code.
5. Đọc và xác nhận các quyết định D-01 đến D-10.
6. Tạo solution .NET và frontend theo roadmap.
7. Bắt đầu với Authentication/Profile hoặc Teams, không làm nhiều module cùng lúc.

Kết luận

Aloball nên được xây dựng như một Modular Monolith có ranh giới nghiệp vụ rõ ràng, ưu tiên quản lý đội bóng và trận đấu trước. Antigravity giúp tăng tốc boilerplate, UI và review; người phát triển cần giữ quyền quyết định về data model, authorization, business logic, testing và release. Mục tiêu MVP là một sản phẩm nhỏ nhưng đáng tin cậy, không phải danh sách tính năng lớn nhưng thiếu kiểm soát.

— Hết tài liệu —